Lab 5: ER diagrams

In this lab you will get practice in designing ER diagrams, translating them to SQL, and understanding the impact of constraints on database design.

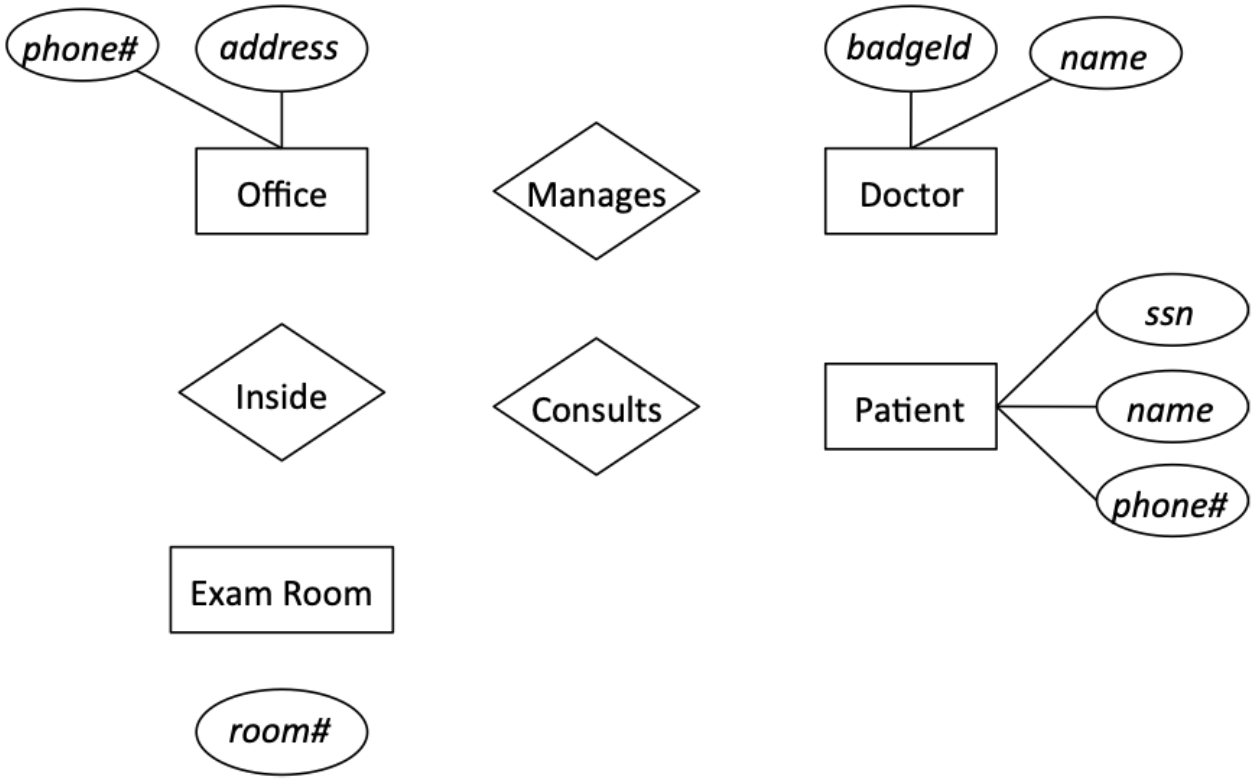
Objectives:

- Read and understand the meaning of ER diagrams.
- Match business rules to ER models.
- Convert ER diagrams to SQL implementation.
- Convert SQL implementation to ER diagram.
- Implement attribute level constraints, tuple level constraints, and update and delete constraints.

Part 1: ER Diagrams

Q1: Your friend is impressed by your skills in 345, and asks you to help her design a database to model doctors' offices across the Pioneer Valley. She started the ER diagram below, which already contains all the entity sets, their attributes and relationships she wants you to consider. You need to finish the diagram and define a number of constraints to ensure that you model the semantics of an office as closely as possible. Make sure that you do not impose additional constraints not defined by the model.

- An office may be managed by at most one doctor. A doctor is uniquely identified by their badgeId, and may manage more than one office. Draw the necessary constraints to capture this requirement.
- An office is identified by its address, and contains one or more exam rooms. An exam room can be identified by its room number and the office that it is in. Capture these constraints by adding to or modifying the diagram where necessary.
- When a patient visits an office, they have a consultation with a doctor in an examination room. We want to store the `date` when a consultation happens.
- Each patient is uniquely identified by their SSN.



Write the appropriate SQL statements to create the tables that correspond to this diagram. Assume that there is no separate table for relationships Manages and Inside. Use the attribute names as shown in the diagram (ignore the character "#") or specified in the description; all attributes are of type VARCHAR(10). If you add an attribute in table A that is a foreign key to table B, use the name of the attribute as it appears in table B. Submit your statements as follows:

Q1a.sql: Write the statement to create the relation that corresponds to the entity set `Doctor`. Submit your statement in file Q1a.sql

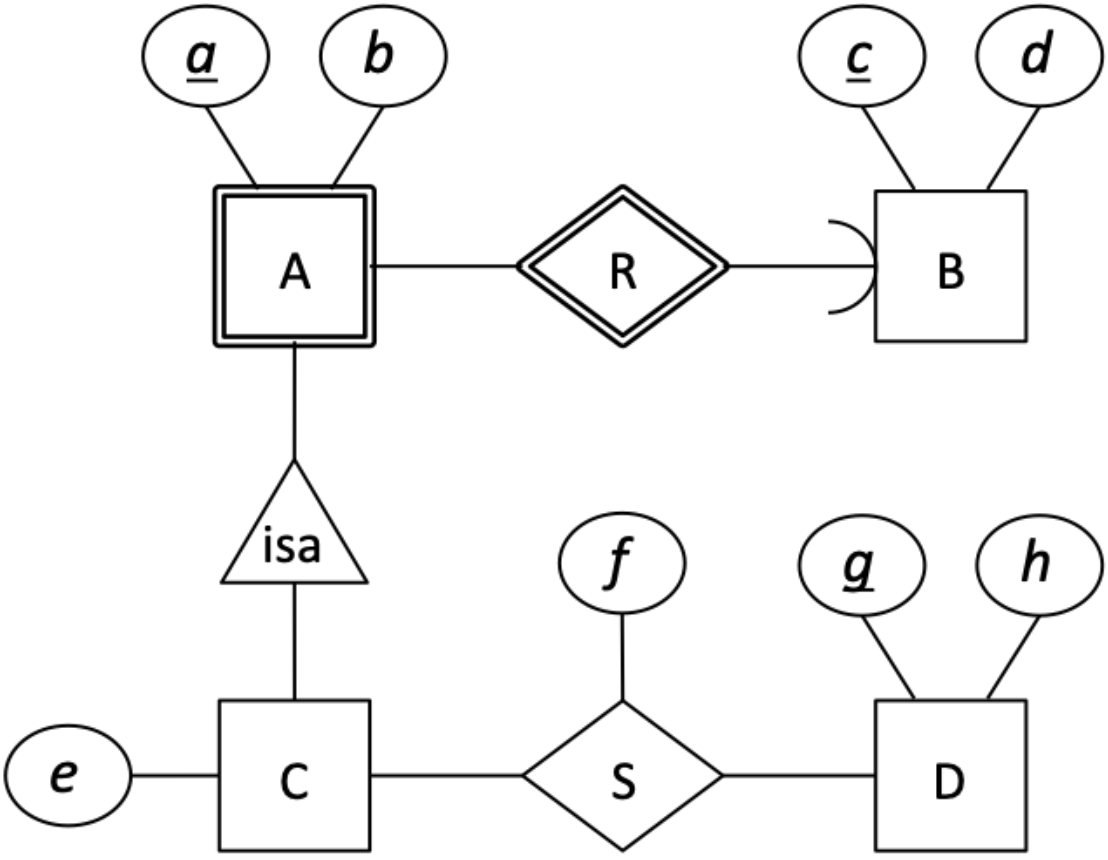
Q1b.sql: Write the statement to create the relation that corresponds to the entity set `Patient`. Submit your statement in file Q1b.sql

Q1c.sql: Write the statement to create the relation that corresponds to the entity set `Office`. Submit your statement in file Q1c.sql

Q1d.sql: Write the statement to create the relation that corresponds to the entity set `ExamRoom`. Submit your statement in file Q1d.sql

Q1e.sql: Write the statement to create the relation that corresponds to the relationship `Consults`. Submit your statement in file Q1e.sql

Q2: Write the SQL DDL commands (CREATE TABLE statements) to convert the ER diagram below to a database schema. Assume all attributes are integers (INT). Pay attention to the constraints you need to declare (primary keys and foreign keys).



Do not create tables for relationships unless they are necessary. Make sure you enter the CREATE TABLE statements in the proper order (e.g., a statement will fail if it is specifying a foreign key to another table that has not been created yet. Submit all statements in file Q2.sql.

Part 2: Constraints

The goal in this part is to understand the differences among 3 types of constraints: primary keys, foreign keys, and unique constraints. We have discussed these constraints before, but now we will get further practice with them. Start postgres and create a new database to practice. Create simple tables and add a few tuples as needed to test and answer the following:

For each type of constraint (PRIMARY KEY / FOREIGN KEY /UNIQUE):

1. Can the constraint be defined on a single attribute? Can the constraint be defined on multiple attributes?
2. Can you have multiple constraints of the same type on the same table? (e.g., two primary keys, or two unique constraints)
3. Can two tuples in the table have the same value for the constraint?
4. Can tuples take NULL values for the constraint?
5. Can constraints of the same or different types overlap? For example, can part of a foreign key be defined as unique? Can two foreign keys overlap? Can the same attribute have a foreign key constraint referencing multiple tables?

Understanding join cardinality with foreign keys

Create the following two simple tables in your practice database and populate them with a few tuples:

```
CREATE TABLE T1 (a int PRIMARY KEY, b int);
CREATE TABLE T2 (c int PRIMARY KEY,
            d int NOT NULL REFERENCES T1(a));
```

Let's say that T1 has x tuples and T2 has y tuples. How many tuples are there in the result of each of the following queries?

```
SELECT * FROM T1, T2 WHERE a=d;
SELECT * FROM T1 LEFT OUTER JOIN T2 ON a=d;
SELECT * FROM T1 RIGHT OUTER JOIN T2 ON a=d;
```

Do you understand how the foreign key impacts the size of the result? Can you reason about the minimum and maximum result size for each of these queries under any random, valid data instance where T1 has x tuples and T2 has y tuples?

We will have polls towards the end of the class session to test your understanding.

Q3: A customer's database has the following tables:

- Product, listing product ids and names.
- Inventory, listing the quantity for each product.
- Supplier, listing the supplier id, name and address.
- Supplies, indicating which supplier (sid) supplies which product (pid).
- PurchaseOrder, indicating the quantity that is on order for each product.

You have the following CREATE TABLE statements, but they are incomplete.

```
CREATE TABLE Product (pid INT PRIMARY KEY,
                      name VARCHAR(20))

CREATE TABLE Inventory (pid INT PRIMARY KEY,
                        quantity INT)

CREATE TABLE Supplier (sid INT PRIMARY KEY,
                       name VARCHAR(20),
                       address VARCHAR(50))

CREATE TABLE Supplies (sid INT,
                       pid INT,
                       PRIMARY KEY (sid, pid))

CREATE TABLE PurchaseOrder (pid INT,
                            quantity INT)
```

Rewrite the statements above to enforce the following constraints:

1. Indicate all the appropriate foreign keys. For example, ensure that a purchase order refers to a product that exists in the Product table.

2. Ensure that the value of the quantity attribute from table Inventory is always greater than or equal to 0.

3. Whenever someone deletes a tuple in Supplier, any tuple in Supplies that referred to it should also be deleted.

Do not change any attribute types, only add the necessary constraints. Submit the revised statements in file Q3.sql.